

SLA-AWARE CLOUD STORAGE RESOURCE ALLOCATION OPTIMIZER

Technical Foundations, Algorithmic Logic, and Conditional Analysis Guide

1. System Foundations & Technical Overview

The **SLA-Aware Cloud Storage Resource Allocation Optimizer** is an automated decision-support tool engineered to solve the complex multi-objective resource allocation problem faced by cloud storage tenants and infrastructure architects. In modern enterprise IT environments, selecting the optimal cloud storage tier is challenging because public cloud providers (such as AWS, Azure, and Google Cloud) offer storage products with fundamentally divergent physical performance profiles, contractual uptime guarantees, and pricing structures. High-performance tiers such as SSD Block Storage deliver ultra-low access latencies (2.0 ms) and high availability guarantees (99.999% uptime), but incur high monthly unit costs (0.15 per GB). Conversely, low-cost Object Storage offers economical pricing (0.02 per GB per month), but imposes higher access latencies (50.0 ms) and lower availability guarantees (99.0%). Without an automated optimization system, storage administrators typically rely on static rules, leading to either costly financial over-provisioning or severe Service Level Agreement (SLA) violations that disrupt business operations.

To eliminate manual guesswork and guarantee SLA compliance, the system bases its allocation decisions on four primary application parameters: Maximum Tolerable Access Latency (L_{req} in milliseconds), Availability SLA Target (A_{req} in percentage), Cost Optimization Weight (α , Alpha), and optional Monthly Budget Constraint (B in USD). The first two parameters represent **hard physical and operational constraints**. Access latency measures hardware responsiveness; if a high-throughput transactional database requires latency under 10 ms, Object Storage (50 ms) is physically incapable of supporting the workload and is immediately disqualified. Similarly, SLA availability defines allowable annual downtime, ranging from 87.6 hours per year at 99.0% availability down to just 5.26 minutes per year at 99.999% ("Five Nines"). Any cloud storage tier providing an uptime guarantee below A_{req} is excluded to prevent contractual non-compliance.

The third parameter, Cost Optimization Weight (α), addresses the economic challenge of balancing monetary expenditure (\$) against uptime reliability (%). Because storage cost is an objective to be minimized while availability is an objective to be maximized, the algorithm first converts availability into an **unavailability fraction**: $U(i) = 1.0 - (A(i) / 100.0)$. This transformation aligns both metrics into a unified minimizability framework. The weight coefficient α (configurable between 0.0 and 1.0) allows infrastructure managers to customize allocation priorities according to organizational strategy. Setting a high Alpha ($\alpha \rightarrow 1.0$) prioritizes aggressive cost reduction, instructing the engine to select the cheapest compliant tier. Setting a low Alpha ($\alpha \rightarrow 0.0$) prioritizes maximum availability, selecting the most resilient tier regardless of cost. Setting $\alpha = 0.5$ establishes an equal balance between cost savings and uptime safety, with companion weight $\beta = 1.0 - \alpha$.

The fourth parameter, Monthly Budget Constraint (B), enforces financial boundaries. Total monthly expenditure for a requested storage capacity S_{req} (in Gigabytes) is calculated as $C(i) = Cost_per_GB(i) \times S_{req}$. If a budget limit B is specified ($B > 0$), any candidate tier whose total monthly cost exceeds B is removed from the feasible solution set. In scenarios where no single storage tier can satisfy all latency, SLA, and budget requirements simultaneously, the system terminates execution safely with an explicit exception dialog. This dialog provides the administrator with details on the closest available alternative, preventing silent system failures and guiding informed constraint adjustments.

System implementation follows a **Three-Tier Software Architecture** that maintains strict separation of concerns across user presentation, business logic, and database persistence. The Presentation Layer is built using Streamlit, providing an intuitive web interface with parameter sliders, workload capacity calculators, dynamic metric cards, and responsive Plotly visual charts. When a request is submitted, parameters flow into the Application Logic Layer (`heuristic.py`), which executes constraint filtering, Min-Max metric scaling, objective scoring, and baseline comparisons. Finally, the Data Layer (`database.py`) persists active cloud tier specifications and automatically records every recommendation transaction into an embedded SQLite database (`storage_allocation.db`) to ensure complete audit compliance for IT governance.

To empirically prove optimization performance, the system simultaneously benchmarks its heuristic recommendations against three traditional baseline algorithms: First Fit (FF), which

selects the first compliant tier in database order; Best Fit (BF), which minimizes excess SLA availability slack; and Worst Fit (WF), which maximizes availability slack. By capturing these baselines alongside the heuristic recommendation, the system provides infrastructure managers with clear empirical evidence of cost savings and SLA compliance across historical workload traces.

2. System Conceptual Architecture & Decision Pipeline

The conceptual architecture defines how data flows sequentially from input capture to optimal tier selection across two core computational phases: **Hard Constraint Pre-Filtering** and **Min-Max Dual-Objective Scoring**. Figure 1 outlines this structural framework.

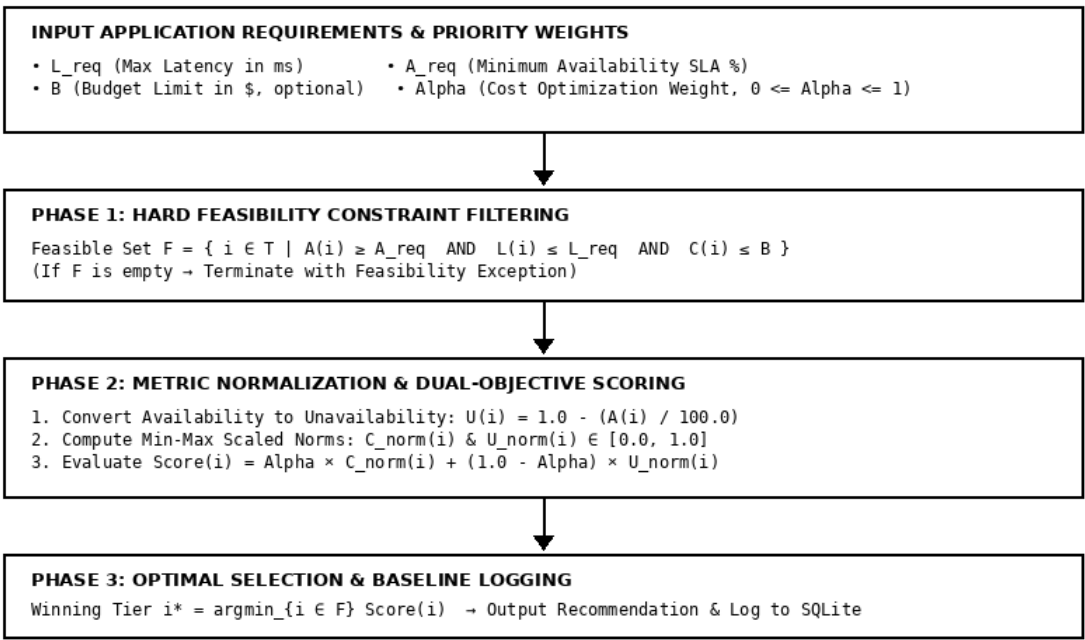


Figure 1: Conceptual Framework showing parameter relationships and decision pipeline

As shown in Figure 1, the pipeline accepts application latency targets, SLA availability bounds, budget ceilings, and priority weights. Phase 1 filters out non-compliant storage tiers to form a feasible candidate set F. Phase 2 converts availability to unavailability, applies Min-Max range scaling to normalize metrics into standard [0.0, 1.0] bounds, and evaluates the weighted dual-objective score. Phase 3 selects the winning tier that minimizes the combined score and logs the complete allocation transaction to the SQLite database.

3. Core Heuristic Algorithm Pseudocode

The core decision engine executes the **SLA-Aware Dual-Objective Storage Allocation Heuristic Algorithm**. Figure 2 provides a formal pseudocode snapshot specifying the algorithmic logic flow.

```
Input: Storage Tiers T, Size S_req, Availability Target A_req, Latency Limit L_req, Budget B, Weight  $\alpha$ 
Output: Optimal Storage Tier i* and Estimated Monthly Cost

1: FeasibleSet F  $\leftarrow \emptyset$ 
2: for each storage tier i in T do
3:   TotalCost[i]  $\leftarrow$  UnitCost[i]  $\times$  S_req
4:   if Availability[i]  $\geq$  A_req and Latency[i]  $\leq$  L_req then
5:     if Budget B is set and TotalCost[i] > B then continue
6:     F  $\leftarrow$  F  $\cup$  {i}
7:   end if
8: end for
9: if F =  $\emptyset$  then return Error("No tier meets SLA, Latency, or Budget bounds")
10: for each tier i in F do
11:   Unavailability[i]  $\leftarrow$  1.0 - (Availability[i] / 100.0)
12: end for
13: Compute Cost_norm(i) and Unavail_norm(i) using Min-Max scaling over F
14: for each tier i in F do
15:   Score[i]  $\leftarrow$   $\alpha \times$  Cost_norm(i) + (1.0 -  $\alpha$ )  $\times$  Unavail_norm(i)
16: end for
17: i*  $\leftarrow$  argmin_{i  $\in$  F} Score[i]
18: return Recommended Tier i*, TotalCost[i*], Availability[i*], Latency[i*]
```

Figure 2: Algorithm 3.1 Pseudocode logic of the core SLA-aware dual-objective heuristic algorithm

Formal Algorithm Step Breakdown:

* **Lines 1–8 (Hard Constraint Filtering):** The algorithm initializes an empty feasible set F. For each storage tier i in database T, it calculates total monthly cost $C(i) = \text{UnitCost}[i] \times S_{\text{req}}$. If tier i satisfies $\text{Availability}[i] \geq A_{\text{req}}$, $\text{Latency}[i] \leq L_{\text{req}}$, and $\text{TotalCost}[i] \leq \text{Budget } B$, it is added to F.

* **Line 9 (Exception Check):** If no storage tier satisfies all constraints ($F = \emptyset$), execution halts with an explicit exception message.

* **Lines 10–12 (Unavailability Conversion):** Availability percentages are transformed into unavailability fractions $U(i) = 1.0 - (\text{Availability}[i] / 100.0)$ for all tiers in F.

* **Line 13 (Min-Max Normalization):** Cost $C(i)$ and unavailability $U(i)$ are normalized to $[0.0, 1.0]$ over feasible set F to eliminate unit disparities.

* **Lines 14–16 (Weighted Dual-Objective Scoring):** Objective score is calculated as $\text{Score}[i] = \alpha \times \text{Cost_norm}(i) + (1.0 - \alpha) \times \text{Unavail_norm}(i)$.

* **Lines 17–18 (Optimal Selection):** The tier i^* that minimizes $\text{Score}[i]$ is selected as the winning recommendation and logged to SQLite.

4. Operational System Algorithm Flowchart

Figure 3 illustrates the complete operational flowchart tracing data movement, constraint verification branches, metric normalization stages, and database persistence operations across the platform.

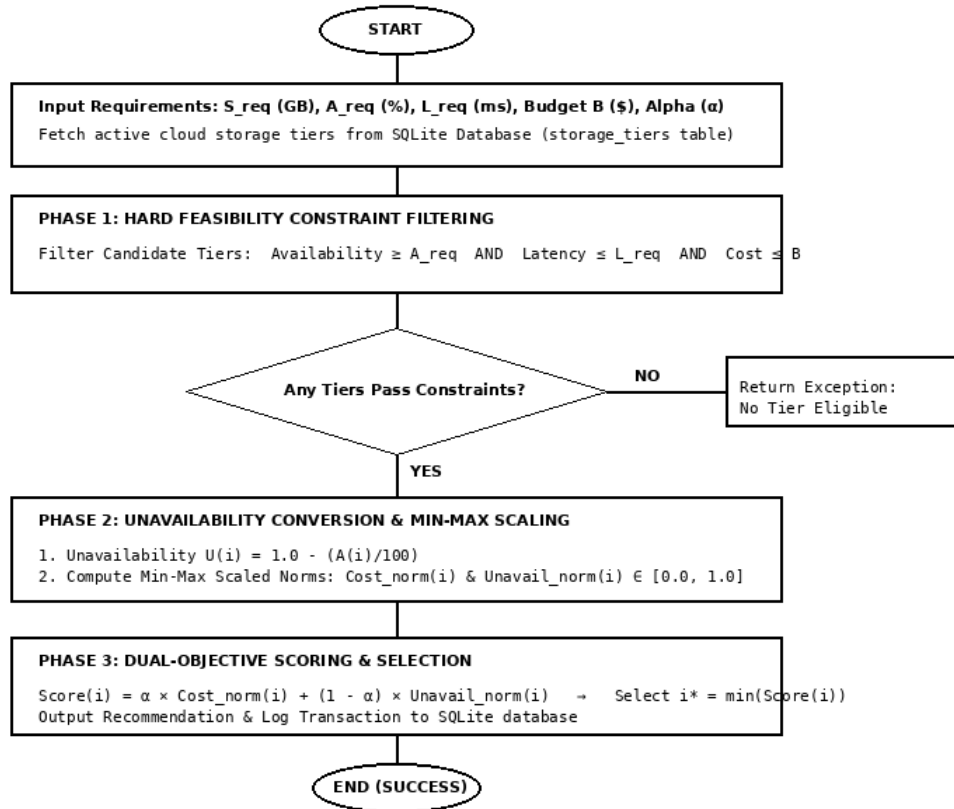


Figure 3: System Algorithm Flowchart showing Constraint Pipeline and Dual Scoring

The flowchart illustrates how user workload requirements pass into the system engine, trigger SQLite database queries to retrieve active tier benchmarks, execute pre-filtering checks, calculate Min-Max scaled scores, select the optimal tier, and write allocation records into the database audit trail.

5. Conditional Logic & Parameter Sensitivity Analysis (If-Then Scenarios)

The behavior of the allocation engine is fully deterministic and changes dynamically based on parameter inputs. The following conditional scenarios explain how changing specific parameter values impacts candidate filtering, metric scoring, and final tier selection:

* **Scenario 1: IF Access Latency Limit (L_{req}) is Changed or Lowered:**

* **IF** L_{req} is set to a strict sub-5.0 ms threshold (e.g., $L_{req} = 5.0$ ms for transactional OLTP DBs), **THEN** File Storage (10.0 ms) and Object Storage (50.0 ms) are physically disqualified during Phase 1 pre-filtering.

* **THEN** Block Storage (2.0 ms) becomes the sole member of feasible set F and is selected automatically regardless of cost or the value of Alpha.

* **IF** L_{req} is relaxed to 50.0 ms (e.g., for background batch processing), **THEN** all candidate storage tiers pass latency pre-filtering, enabling multi-objective scoring across all options.

* **Scenario 2: IF Availability SLA Target (A_{req}) is Changed or Raised:**

* **IF** A_{req} is raised to "Five Nines" ($A_{req} = 99.999\%$ for mission-critical systems), **THEN** Object Storage (99.0%) and File Storage (99.99%) fail the uptime check and are disqualified.

* **THEN** Block Storage (99.999%) is selected as the only compliant tier.

* **IF** A_{req} is set to 99.0% (e.g., for dev/test environments), **THEN** Object, File, and Block storage tiers all pass reliability pre-filtering, allowing cost weight Alpha to determine the winner.

* **Scenario 3: IF Cost Weight (α , Alpha) is Changed:**

* **IF** Alpha is shifted to extreme cost focus ($\alpha = 1.0$, $\beta = 0.0$), **THEN** unavailability is assigned 0% weight, and the algorithm selects the cheapest compliant tier (e.g., Object Storage at \$0.02/GB if it passes latency and SLA limits).

* **IF** Alpha is shifted to extreme availability focus ($\alpha = 0.0$, $\beta = 1.0$), **THEN** cost is assigned 0% weight, and the algorithm selects the tier with the highest SLA uptime (e.g., Block Storage at 99.999%).

* **IF** Alpha is set to balanced weighting ($\alpha = 0.5$, $\beta = 0.5$), **THEN** equal weight is given to normalized cost and normalized unavailability, selecting the optimal middle tier (e.g., File Storage).

* **Scenario 4: IF Monthly Budget Constraint (B) is Imposed or Reduced:**

* **IF** a budget limit B is specified (e.g., $B = 30.00$ for a 500 GB workload), **THEN** Block Storage (75.00/mo) and File Storage (\$40.00/mo) are disqualified for exceeding budget.

* **IF** Object Storage (\$10.00/mo) passes latency and SLA checks, **THEN** Object Storage wins as the only affordable tier.

* **IF** budget B is reduced below all tier costs (e.g., $B = \$5.00$), **THEN** feasible set F becomes empty ($F = \emptyset$), **THEN** the algorithm terminates safely with an explicit exception dialog detailing closest available options.

* **Scenario 5: IF Storage Capacity (S_req) Scales Up:**

* **IF** requested storage capacity increases from 100 GB to 1,000 GB, **THEN** total monthly cost for each tier scales up linearly ($C(i) = \text{Cost_per_GB}(i) \times S_{\text{req}}$).

* **THEN** tiers that were affordable under a small request may exceed budget ceilings B under larger requests, shifting tier eligibility dynamically.

6. Simple Step-by-Step Optimization Process

The algorithm follows seven simple steps to generate a recommendation:

* **Step 1 (Sizing):** Determine the required storage capacity in Gigabytes (S_{req}).

* **Step 2 (Cost Calculation):** Calculate total monthly cost for each storage tier: $\text{Cost} = \text{UnitCost} \times S_{\text{req}}$.

* **Step 3 (Hard Filtering):** Remove any tier that fails access latency ($\text{Latency} > L_{\text{req}}$), uptime ($\text{SLA} < A_{\text{req}}$), or budget ($\text{Cost} > B$).

* **Step 4 (Unavailability Conversion):** Convert uptime percentage into unavailability:

$\text{Unavailability} = 1.0 - (\text{SLA} / 100).$

* **Step 5 (Range Scaling):** Normalize cost and unavailability into a standard 0.0 to 1.0 scale over eligible tiers.

* **Step 6 (Dual Scoring):** Calculate final score: $\text{Score} = \text{Alpha} \times \text{Cost_norm} + (1.0 - \text{Alpha}) \times \text{Unavail_norm}.$

* **Step 7 (Selection):** Select the tier with the lowest score as the winning recommendation and save the record to SQLite.

7. Simple Worked Example (500 GB Request)

Here is a simple numerical walk-through demonstrating how the algorithm selects a tier:

Input Requirements:

* **Storage Size:** 500 GB

* **Availability Target:** 99.0% minimum uptime

* **Latency Limit:** 20.0 ms maximum delay

* **Cost Priority (Alpha):** 0.7 (70% weight on cost savings, 30% weight on uptime reliability)

Candidate Tiers in Database:

1. **Block Storage (SSD):** 0.15/GB (75.00/month), 99.999% SLA, 2.0 ms latency.

2. **File Storage (NAS):** 0.08/GB (40.00/month), 99.99% SLA, 10.0 ms latency.

3. **Object Storage (S3):** 0.02/GB (10.00/month), 99.0% SLA, 50.0 ms latency.

Step-by-Step Execution:

1. **Constraint Check:** Object Storage latency is 50.0 ms, which exceeds the 20.0 ms limit →

Object Storage is disqualified. Block Storage (2.0 ms) and File Storage (10.0 ms) pass.

2. **Metric Normalization:** Over eligible tiers (Block & File), Block Storage has higher cost ($\text{Cost_norm} = 1.0$) and lower unavailability ($\text{Unavail_norm} = 0.0$). File Storage has lower cost ($\text{Cost_norm} = 0.0$) and higher unavailability ($\text{Unavail_norm} = 1.0$).

3. Score Evaluation (Alpha = 0.7, Beta = 0.3):

* Block Storage Score = $(0.7 \times 1.0) + (0.3 \times 0.0) = \mathbf{0.70}$

* File Storage Score = $(0.7 \times 0.0) + (0.3 \times 1.0) = \mathbf{0.30}$

4. Final Decision: File Storage has the lower score (0.30 vs 0.70) → **File Storage is selected as the winning recommendation!**